TECH EVENT TICKET BOOKING SYSTEM

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

M.Tejaswani (VTU25564)

10211CS224

Full Stack Application Development
Slot: E3-B19

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled **"Tech Event Ticket Booking"** by **M.Tejaswani (VTU25564)** has been carried out in Winter semester of Academic Year 2025–2026 (WS2526).

Signature of Faculty
Dr.X.Arogya Preskilla
Assistant Professor(Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
May, 2026

Signature of Course Coordinator
Dr. M. Sumithra
Assistant Professor (Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(M.Tejaswani)

Date: 06/05/2026

ABSTRACT

The Tech Event Ticket Booking system is a comprehensive full-stack web application designed to streamline event management and ticketing within an organisation. This system enables users to browse, book, and manage event tickets seamlessly while allowing administrators to create and manage events efficiently. The application implements modern web technologies including React.js for the frontend, Node.js with Express.js for the backend, and MySQL for data persistence. Two key features have been added to enhance user engagement: Real-time Notifications using WebSocket (Socket.io) technology for instant updates on bookings and events, and an Event Categorisation System using Tags that allow users to filter and discover events based on their interests. The system employs JWT-based authentication for security and follows a modular architecture pattern. This report details the development process, implementation strategies, database design, and integration of advanced features for real-time communication.

Keywords: Full Stack Development, Event Management, WebSocket, Real-time Notifications, React.js, Node.js, MySQL, JWT Authentication, Event Tagging System

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

JWT	JSON Web Token
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
SQL	Structured Query Language
ORM	Object Relational Mapping
SPA	Single Page Application
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
WS	WebSocket
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

The management of technical events has become increasingly complex with growing attendance and diverse event types. Traditional methods of event booking through manual registration or email often lead to inefficiencies, double bookings, and poor user experience. The Tech Event Ticket Booking system addresses these challenges by providing a digital platform that simplifies event discovery, ticket booking, and administrative management.

This system serves as a centralised hub where employees and students can view upcoming events, filter by categories or tags, book tickets, and receive real-time notifications about their bookings and new events. Administrators gain the ability to create events, manage attendee lists, track ticket availability, and communicate with users through the notification system.

The application leverages modern web technologies to ensure scalability, real-time communication, and an intuitive user interface. The integration of WebSocket technology enables instant notifications, while the tag-based categorisation system improves event discoverability.

1.2 Aim of the Project

The primary aim of this project is to develop a complete event management solution that:

1. Provides a user-friendly interface for browsing and booking event tickets
2. Enables administrators to create, update, and manage events efficiently
3. Implements a real-time notification system for booking confirmations and event updates
4. Allows users to filter and discover events using tags and categories

5. Ensures data security through JWT-based authentication
6. Supports payment processing for ticket reservations
7. Generates reports and analytics for event management

1.3 Scope of the Project

The scope of this project includes:

1. User authentication and authorisation with role-based access control
2. Event CRUD operations for administrators
3. Ticket booking and cancellation functionality
4. Real-time notification system using WebSocket
5. Tag-based event categorisation and filtering
6. User dashboard for viewing bookings and notifications
7. Admin dashboard for event and booking management
8. Email notifications for booking confirmations
9. Calendar view for event visualisation
10. Responsive design for mobile and desktop devices

1.4 Framework

The project utilises the MERN-style stack (MongoDB replaced with MySQL) along with Socket.io for real-time communication.

Frontend Framework: React.js 18.3.1 with Vite

- React Router v6 for client-side routing
- Axios for HTTP requests

- Bootstrap 5.3.3 for responsive UI

Backend Framework: Node.js with Express.js

- Express v4.19.2 for REST API
- Socket.io for WebSocket communication
- JWT for authentication
- Nodemailer for email notifications

Database: MySQL 8.0

- Structured relational database
- Normalisation for data integrity

Authentication: JWT (JSON Web Tokens)

- Secure token-based authentication
- Role-based access control (User, Admin)

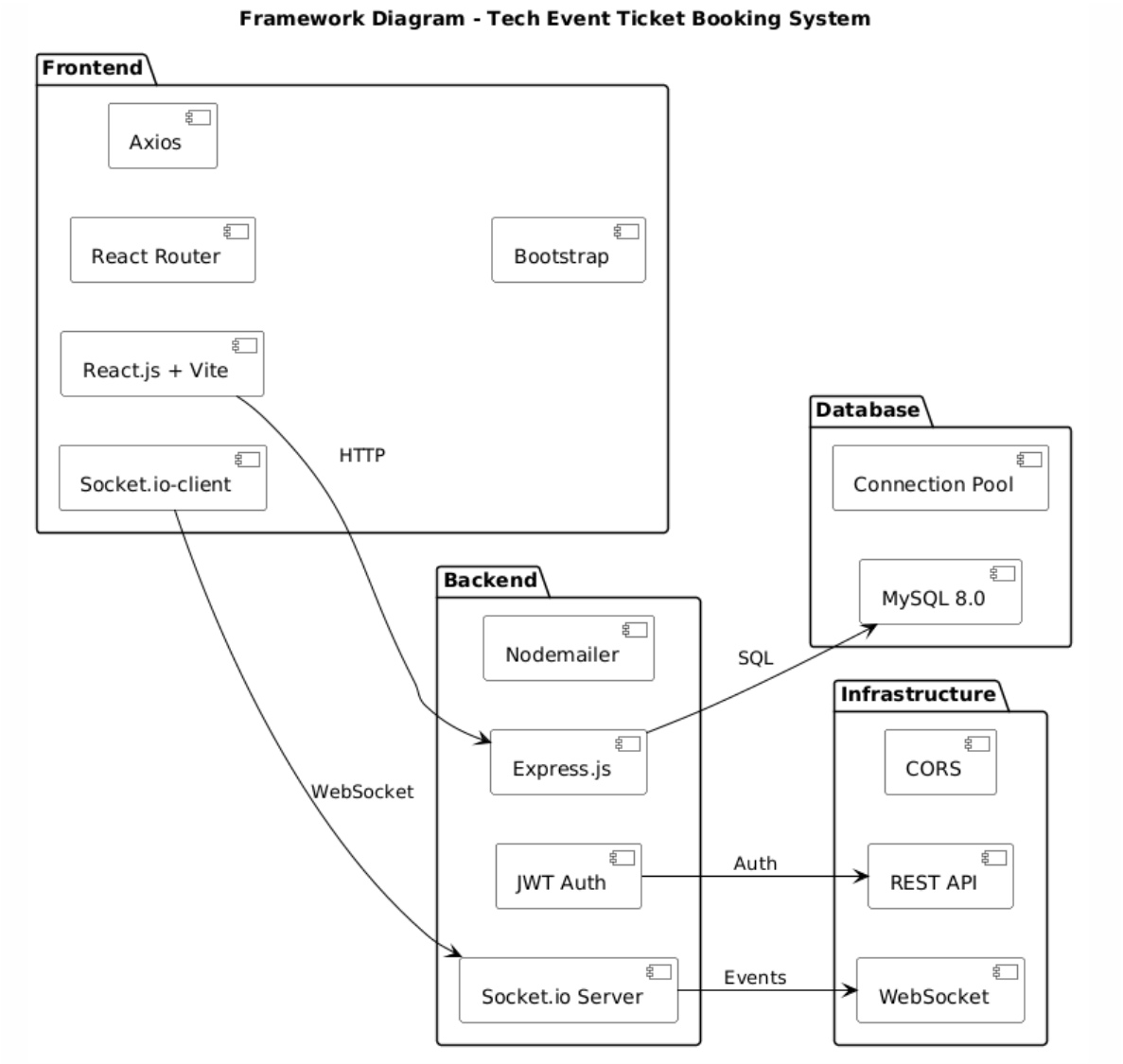

Figure 1.1: Framework Diagram – Tech Event Ticket Booking System

Figure 1.1 illustrates the explanation of the framework diagram for the Tech Event Ticket Booking System, which follows a four-layer architecture consisting of the frontend, backend, database, and real-time layers. The frontend layer provides the user interface and manages user interactions using technologies such as React.js, Vite, and Bootstrap, while also communicating with the backend through REST APIs and WebSocket connections. The backend layer acts as the core of the system, handling business logic, authentication, and request processing using Node.js and Express.js, and it interacts with the database for storing and retrieving data. The database layer, implemented using MySQL, stores all persistent data including user details, events, bookings, and payments, ensuring data integrity through a normalized schema and efficient access via connection pooling. The real-time layer enables instant communication by managing WebSocket connections, allowing features like live updates, booking confirmations, and notifications. Overall, this layered architecture ensures efficient data flow, scalability, maintainability, and real-time responsiveness in the system.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

Several existing applications provide event management and ticketing solutions.

1. Eventbrite (`https://www.eventbrite.com/`)

Eventbrite is a leading global platform for event management and ticketing. It offers comprehensive event discovery, ticket sales, and attendee management features. However, it is designed for public events and lacks customisation for institution-specific technical event requirements.

2. Meetup (`https://www.meetup.com/`)

Meetup focuses on community event organisation and social networking around events. While it provides good event discovery features, it is not designed for academic or technical event workflows with administrative controls.

3. Ticketmaster (`https://www.ticketmaster.com/`)

Ticketmaster specialises in large-scale entertainment events and ticketing. It is enterprise-level but not suitable for smaller technical events due to complexity and cost.

4. Eventsquid (`https://www.eventsquid.com/`)

Eventsquid provides event management for conferences and meetings. It includes features like attendee management and reporting but lacks real-time notification capabilities.

Comparison with Our Solution:

- Lightweight and customisable for technical event use
- Real-time WebSocket-based notifications
- Tag-based event categorisation for better filtering
- Role-based access control specific to organisational needs
- Integration with internal email systems

- Cost-effective with open-source technologies
- User-friendly interface for students and faculty
- Faster performance with optimized architecture
- Custom workflows for approvals and registrations
- Real-time seat availability tracking
- Secure JWT-based authentication
- Automated email notifications and reminders
- Scalable design for multiple departments
- Detailed analytics and reporting features
- Centralized admin dashboard
- Easy integration with ERP or student portals

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is built using React.js with Vite as the build tool. Installation steps:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios react-router-dom bootstrap socket.io-client
```

Listing 3.1: Frontend environment setup

Lisitng 3.1 environment setup for the frontend is carried out using React.js with Vite as the build tool to ensure fast development and efficient performance. Vite simplifies project initialization and provides features such as hot module replacement, which enhances the development experience. The setup process begins by creating a new React project using Vite, followed by installing the required dependencies. Additional libraries such as Axios are used for handling API requests, React Router DOM enables client-side routing, Bootstrap is included for responsive and user-friendly interface design, and Socket.io-client supports real-time communication. This setup provides a strong foundation for building a scalable, interactive, and high-performance frontend application.

Dependencies installed:

- React 18.3.1
- React Router DOM 6.25.1
- Axios 1.7.2 for API calls
- Bootstrap 5.3.3 for styling

- Socket.io-client for WebSocket communication

3.1.2 Project Structure

The frontend project follows a component-based architecture:

```
frontend/
+-- src/
|   +-- components/
|   |   +-- AppNavbar.jsx
|   |   +-- EventCard.jsx
|   |   +-- EventFilters.jsx
|   |   +-- NotificationCenter.jsx
|   |   +-- TagBadge.jsx
|   |   +-- TagFilter.jsx
|   +-- contexts/
|   |   +-- AuthContext.jsx
|   |   +-- ThemeContext.jsx
|   |   +-- ToastContext.jsx
|   +-- pages/
|   |   +-- HomePage.jsx
|   |   +-- LoginPage.jsx
|   |   +-- DashboardPage.jsx
|   |   +-- EventDetailsPage.jsx
|   +-- services/
|   |   +-- api.js
|   |   +-- notificationService.js
|   |   +-- tagService.js
|   |   +-- eventService.js
|   +-- App.jsx
+-- package.json
+-- vite.config.js
```

Listing 3.2: Frontend project structure

Listing 3.2 frontend project follows a component-based architecture, which improves modularity, reusability, and maintainability of the application. The main source code is organized inside the src directory, where different functionalities are separated into dedicated folders. The components folder contains reusable UI elements such as navigation bars, event cards, filters, and notification components, which help in building a consistent user interface. The contexts folder manages global

state using React Context API, handling features like authentication, theming, and toast notifications across the application.

3.1.3 Notification Service Implementation

The NotificationService handles WebSocket connections and API calls:

```
import { io } from 'socket.io-client';
const socket = io('http://localhost:5000');

export const notificationService = {
  initializeSocket: (userId) => {
    if (!socket.connected) {
      socket.emit('join_notifications', userId);
    }
  },

  onNotification: (callback) => {
    socket.on('notification', callback);
  },

  generateTestNotifications: async () => {
    const response = await axios.post(
      'http://localhost:5000/api/notifications/test/generate',
      {},
      { headers: { Authorization: `Bearer ${token}` } }
    );
    return response.data;
  }
};
```

Listing 3.3: notificationService.js – Real-time notification handling

Lisitng 3.3 notification Service is responsible for handling real-time communication and notification management within the application. It utilizes the Socket.io-client library to establish a WebSocket connection with the backend server, enabling instant data exchange. Once the connection is initialized, the service allows users to join specific notification channels using their user ID, ensuring that messages are delivered only to the intended recipients. This approach improves efficiency and personalization in delivering updates such as booking confirmations and alerts.

3.1.4 Tag Filter Component

Component for filtering events by tags:

```
import React, { useState, useEffect } from 'react';
import { tagService } from '../services/tagService';

const TagFilter = ({ onTagsChange, selectedTags = [] }) => {
  const [tags, setTags] = useState([]);

  useEffect(() => {
    const loadTags = async () => {
      const tagsData = await tagService.getAllTags();
      setTags(tagsData);
    };
    loadTags();
  }, []);

  const handleTagToggle = (tagId) => {
    const newSelectedTags = selectedTags.includes(tagId)
      ? selectedTags.filter((id) => id !== tagId)
      : [...selectedTags, tagId];
    onTagsChange(newSelectedTags);
  };

  return (
    <div className="tag-filter mb-3">
      <label className="form-label fw-bold">Filter by Tags:</label>
      <div className="d-flex flex-wrap gap-2">
        {tags.map((tag) => (
          <button
            key={tag.id}
            className="btn btn-sm"
            onClick={() => handleTagToggle(tag.id)}
            style={{ backgroundColor: tag.color }}
          >
            {tag.name}
          </button>
        ))}
      </div>
    </div>
```

```
38    );
39  };
40
41  export default TagFilter;
```

Listing 3.4: TagFilter.jsx – Event filtering by tags

Listing 3.4 the Tag Filter component is designed to provide an interactive way for users to filter events based on specific tags or categories. It is implemented as a functional React component using hooks such as useState and useEffect to manage state and lifecycle behavior. When the component is loaded, it fetches all available tags from the backend using the tagService and stores them in the local state. This ensures that the filtering options are dynamically populated and always up to date with the database.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built using Node.js with Express.js:

```
1 mkdir backend && cd backend
2 npm init -y
3 npm install express cors mysql2 dotenv jsonwebtoken socket.io
4 npm install --save-dev nodemon
```

Listing 3.5: Backend environment setup

Listing 3.5 backend environment is developed using Node.js with Express.js as the core framework, providing a robust and scalable server-side architecture. The setup process begins by creating a dedicated backend directory and initializing a Node.js project using npm. Essential dependencies are then installed to support various functionalities of the application. Express.js is used to build RESTful APIs and handle routing, while CORS is enabled to allow secure communication between the frontend and backend. The MySQL2 library is used for efficient database connectivity and query execution.

Main dependencies:

- Express.js 4.19.2 for server framework
- Socket.io for real-time communication

- MySQL2 for database connection
- JWT for authentication
- Dotenv for environment variables

3.2.2 WebSocket Server Configuration

```
const http = require("http");
const app  = require("./app");
const { Server } = require("socket.io");

const server = http.createServer(app);

const io = new Server(server, {
  cors: { origin: "http://localhost:5173", credentials: true }
});

const connectedUsers = new Map();

io.on("connection", (socket) => {
  console.log(`User connected: ${socket.id}`);

  socket.on("join_notifications", (userId) => {
    socket.join(`user_${userId}`);
    connectedUsers.set(userId, socket.id);
  });

  socket.on("disconnect", () => {
    for (let [userId, socketId] of connectedUsers.entries()) {
      if (socketId === socket.id) {
        connectedUsers.delete(userId);
        break;
      }
    }
  });
});

app.locals.io = io;
server.listen(5000, () =>
```

```
  console.log("Server running on http://localhost:5000")
);
```

Listing 3.6: server.js – WebSocket initialisation

3.2.3 Notification Controller

```
exports.createTestNotifications = async (req, res) => {
  const userId = req.user.id;
  const notificationTypes = [
    { type: "booking_confirmed", title: "Booking Confirmed",
      message: "Your booking has been confirmed" },
    { type: "event_reminder",   title: "Event Reminder",
      message: "Don't miss your upcoming event" },
    { type: "event_created",    title: "New Event",
      message: "Check out our new event" }
  ];

  const createdNotifications = [];

  for (let i = 0; i < 5; i++) {
    const randomNotif =
      notificationTypes[
        Math.floor(Math.random() * notificationTypes.length)
      ];
    const notifId = await notificationModel.createNotification(
      userId, randomNotif.type, randomNotif.title, randomNotif.message
    );
    createdNotifications.push({ id: notifId, ...randomNotif });
  }

  const io = req.app.locals.io;
  if (io) {
    createdNotifications.forEach((notif) => {
      io.to(`user_${userId}`).emit("notification", {
        type: notif.type, title: notif.title, message: notif.message
      });
    });
  }
```

```
33
34    res.json({
35      success: true,
36      message: `Created ${createdNotifications.length} notifications`,
37      data: createdNotifications
38    });
39  };
```

Listing 3.7: notificationController.js – Test notification generation

Listing 3.7 webSocket server configuration is implemented using Socket.io to enable real-time communication between the server and connected clients. An HTTP server is first created using the Express application, and Socket.io is then attached to this server with proper CORS settings to allow requests from the frontend. This setup ensures secure and seamless communication between different layers of the application. The server listens for incoming client connections and logs each connection using a unique socket ID, which helps in tracking active users.

3.2.4 Tag Routes and Controller

```
1  // Public routes
2  router.get("/",              tagController.getAllTags);
3  router.get("/:tagId/events", tagController.getEventsByTag);
4
5  // Admin-only routes
6  router.post(
7    "/",
8    authMiddleware, adminMiddleware,
9    [body("name").notEmpty().withMessage("Tag name required")],
10   tagController.createTag
11 );
12 router.put(    "/:tagId", authMiddleware, adminMiddleware,
13                tagController.updateTag);
14 router.delete( "/:tagId", authMiddleware, adminMiddleware,
15                tagController.deleteTag);
```

Listing 3.8: tagRoutes.js – Tag API endpoints

Listing 3.8 tag API endpoints are designed to manage event categorization and filtering within the system. These endpoints are divided into public and admin-only routes to ensure proper access control.

The public routes allow all users to retrieve available tags and view events associated with a specific tag. This enables users to easily browse and filter events based on their interests without requiring authentication.

3.2.5 Event Controller with Tag Support

```
async function listEvents(req, res, next) {
  const { page, limit, offset } = parsePagination(req.query);

  const result = await getEvents({
    search:   req.query.search,
    venue:    req.query.venue,
    category: req.query.category,
    dateFrom: req.query.dateFrom,
    dateTo:   req.query.dateTo,
    page, limit, offset
  });

  const eventsWithTags = await Promise.all(
    result.rows.map(async (event) => {
      const tags = await tagModel.getEventTags(event.id);
      return { ...event, tags };
    })
  );

  return res.json({
    success: true,
    data: eventsWithTags,
    pagination: createPaginationMeta(result.total, page, limit)
  });
}
```

Listing 3.9: eventController.js – Event listing with tags

Listing 3.9 event Controller with tag support is responsible for retrieving and delivering event data along with associated tags in a structured format. The controller function processes incoming requests by extracting query parameters such as search keywords, venue, category, date range, and pagination details. These parameters are passed to the event service layer to fetch filtered event records from the database. Pagination is handled efficiently using helper functions to limit the number of results and

improve performance.

3.3 Database Implementation

3.3.1 Database Configuration

```
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host:              process.env.DB_HOST     || "localhost",
  user:              process.env.DB_USER     || "root",
  password:          process.env.DB_PASSWORD || "",
  database:          process.env.DB_NAME     || "event_booking_system",
  waitForConnections: true,
  connectionLimit:   10,
  queueLimit:        0
});

module.exports = pool;
```

Listing 3.10: config/db.js – MySQL connection pool

Listing 3.10 database configuration is implemented using the MySQL2 library with promise support to enable efficient and asynchronous database operations. A connection pool is created to manage multiple database connections, improving performance and scalability by reusing existing connections instead of creating new ones for every request. The configuration parameters such as host, user, password, and database name are retrieved from environment variables using dotenv, ensuring security and flexibility across different deployment environments.

3.3.2 Database Schema

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  role        ENUM('user','admin') DEFAULT 'user',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE events (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  title       VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  category    VARCHAR(60),
  date        DATETIME NOT NULL,
  venue       VARCHAR(255) NOT NULL,
  seats       INT DEFAULT 0,
  image_url   TEXT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE tags (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL UNIQUE,
  color      VARCHAR(7)   DEFAULT '#007bff',
  created_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE event_tags (
  event_id INT NOT NULL,
  tag_id   INT NOT NULL,
  PRIMARY KEY (event_id, tag_id),
  FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
  FOREIGN KEY (tag_id)   REFERENCES tags(id)   ON DELETE CASCADE
);

CREATE TABLE notifications (
  id               INT AUTO_INCREMENT PRIMARY KEY,
  user_id          INT NOT NULL,
  type             VARCHAR(50)  NOT NULL,
  title            VARCHAR(255) NOT NULL,
  message          TEXT NOT NULL,
  related_event_id INT NULL,
  is_read          BOOLEAN   DEFAULT FALSE,
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

```
);

CREATE TABLE bookings (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  user_id         INT NOT NULL,
  event_id        INT NOT NULL,
  tickets_count   INT NOT NULL,
  payment_status VARCHAR(20) DEFAULT 'paid',
  status          VARCHAR(20) DEFAULT 'confirmed',
  created_at      TIMESTAMP   DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id)  REFERENCES users(id)  ON DELETE CASCADE,
  FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE
);
```

Listing 3.11: Core MySQL schema

Listing 3.11 database schema is designed using a normalized relational structure to ensure data integrity, consistency, and efficient querying. The system includes multiple tables such as users, events, tags, event$_tags, notifications, and bookings, each serving a specific purpose. The user stable stores user cred$
$based access control, while the event stable maintains details about technical events including title, descripti$
$to-many relationship between events and tags, enabling flexible event filtering.$

3.3.3 Tag Model Implementation

```
async function getAllTags() {
  const [tags] = await pool.query(
    `SELECT id, name, color, created_at FROM tags ORDER BY name`
  );
  return tags;
}

async function addTagToEvent(eventId, tagId) {
  const [result] = await pool.query(
    `INSERT IGNORE INTO event_tags (event_id, tag_id) VALUES (?, ?)`,
    [eventId, tagId]
  );
  return result.affectedRows > 0;
}

```

```
async function getEventTags(eventId) {
  const [tags] = await pool.query(
    `SELECT t.id, t.name, t.color
     FROM tags t
     INNER JOIN event_tags et ON t.id = et.tag_id
     WHERE et.event_id = ?`,
    [eventId]
  );
  return tags;
}

async function updateEventTags(eventId, tagIds) {
  await pool.query(
    "DELETE FROM event_tags WHERE event_id = ?", [eventId]
  );
  for (const tagId of tagIds) {
    await addTagToEvent(eventId, tagId);
  }
  return true;
}
```

Listing 3.12: tagModel.js – Tag database operations

Listing 3.12 Tag Model implementation handles all database operations related to tags and their association with events. It uses asynchronous functions with MySQL connection pooling to ensure efficient and non-blocking database interactions. The function to retrieve all tags fetches tag details such as ID, name, color, and creation time, ordered alphabetically for better usability. This supports dynamic population of tag data in the frontend.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The application follows a three-tier architecture pattern:

- **Presentation Tier:** React.js frontend with responsive UI components
- **Business Logic Tier:** Node.js/Express.js REST API with authentication
- **Data Tier:** MySQL database with normalised schema
- **Real-time Layer:** Socket.io WebSocket layer for notifications

Authentication flow uses JWT tokens issued on login and verified on protected routes. The WebSocket connection establishes user-specific notification rooms for targeted message delivery.

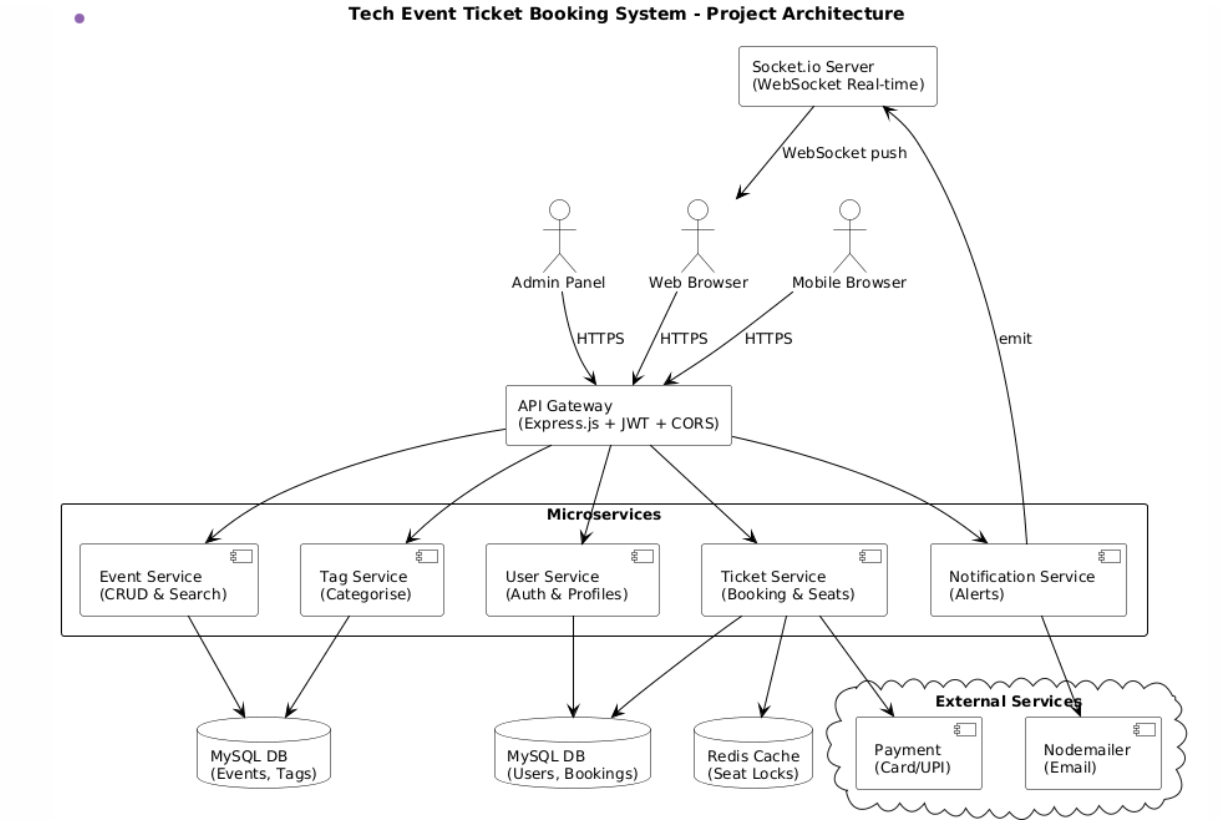

Figure 4.1: Project Architecture – Tech Event Ticket Booking System

The Figure 4.1 illustrates the project architecture of the Tech Event Ticket Booking System, which is organized into multiple tiers to ensure scalability, modularity, and efficient communication. The client tier includes web browsers, mobile browsers, an admin panel, and Socket.io clients that provide user interaction and real-time connectivity. These clients communicate with the API gateway, implemented using Express.js, which acts as a central entry point handling request routing, JWT-based authentication, rate limiting, and CORS policies. This layer ensures secure and structured communication between the frontend and backend services.

4.2 Data Flow Diagram

The application flow follows this pattern:

1. User registers/logs in via authentication API
2. JWT token is issued and stored in the browser
3. WebSocket connection established with user ID
4. User browses events filtered by category/tags

5. User books tickets through booking API

6. Backend creates notification record and emits via WebSocket

7. Notification appears in real-time in notification centre

8. User can mark notifications as read or delete them

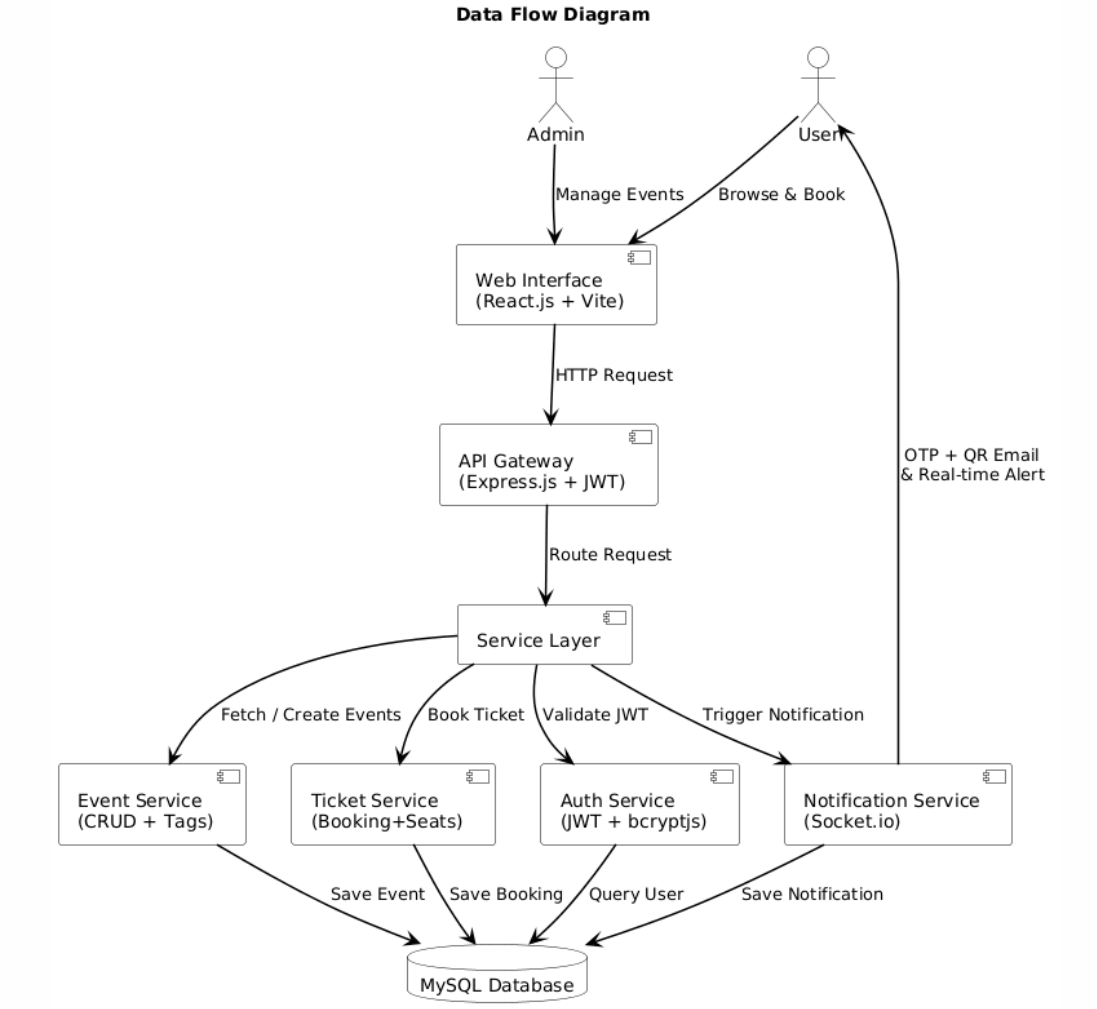

Figure 4.2: Data Flow Diagram – Tech Event Ticket Booking System

The Figure 4.2 presents the data flow of the Tech Event Ticket Booking System, illustrating how user actions are processed step by step across different services. The process begins when a user interacts with the system through a browser to search for or register for events. The request is first handled by the API Gateway, which performs JWT verification and rate limiting to ensure secure access. It then forwards the request to the Authentication Service for login or registration, where user credentials are validated and tokens are issued, with user data stored in the MySQL database. Once authenticated, the request moves to the Event Service, where users can browse and filter events using categories or tags, supported by Elasticsearch for efficient searching.

4.3 Module 1: Authentication Module

Handles user registration, login, and JWT token management:

- User registration with email validation
- Password hashing with bcryptjs
- JWT token generation on login
- Role-based access control (User/Admin)
- Protected route middleware for API endpoints

4.4 Module 2: Event Management Module

Core event operations including creation, listing, and filtering:

- Create events (admin only)
- List events with pagination
- Filter by category, date, venue, and tags
- Update and delete events
- Track available seats
- Event details with associated tags

4.5 Module 3: Notification and Real-time Communication Module

WebSocket-based real-time notifications:

- WebSocket connection management with Socket.io
- User-specific notification rooms
- Booking confirmation notifications

- Event creation announcements
- Notification history storage
- Mark as read/unread and delete functionality

4.6 Module 4: Tag-based Categorisation Module

Event categorisation using a flexible tag system:

- Create and manage tags with colour coding
- Assign multiple tags to events
- Filter events by selected tags
- Admin panel for tag management

Advantages of this Project

a) Real-time Communication: WebSocket integration enables instant notifications for users about booking confirmations and new events, improving user engagement and reducing response time.

b) Flexible Tag System: The tag-based categorisation allows users to discover events based on their interests rather than rigid categories.

c) Scalable Architecture: Modular design and separation of concerns allow easy addition of new features and independent scaling.

d) Security: JWT-based authentication with role-based access control ensures secure user data and admin-only operations.

e) User Experience: Responsive design with real-time feedback provides an intuitive and engaging user interface.

Disadvantages

- WebSocket connections require persistent server resources

- Initial setup and deployment complexity is higher than simple CRUD applications
- Database optimisation required for large-scale data
- Real-time features may be overkill for small organisations

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Features Implemented

5.1.1 Event Management

- Create, read, update, and delete events
- Event listing with pagination
- Search events by title and description
- Filter events by date, venue, and category
- Track available seats; event image support

5.1.2 User Authentication

- User registration and login
- JWT token-based authentication
- Password hashing with bcryptjs
- Role-based access control (User/Admin)

5.1.3 Real-time Notifications (WebSocket)

- WebSocket connection using Socket.io
- User-specific notification rooms
- Real-time booking confirmations and event announcements

- Notification history with persistent storage
- Mark notifications as read/unread; delete individual notifications

5.1.4 Tag-based Event Categorisation

- Create and manage tags with custom colours
- Assign multiple tags to events; filter events by tags
- Admin panel for tag management
- Pre-defined tags: Webinar, Workshop, Seminar, Conference, Networking, Training

5.1.5 Ticket Booking System

- Book tickets for events; track booked tickets per user
- Cancel bookings with seat restoration
- Payment method support (Card/UPI); automatic email confirmation

5.2 API Endpoints

Table 5.1: Tag API Endpoints

Method	Endpoint	Description
GET	/api/notifications	Get user notifications
GET	/api/notifications/unread-count	Get unread count
POST	/api/notifications/test/generate	Generate test notifications
PUT	/api/notifications/:id/read	Mark as read
PUT	/api/notifications/read-all	Mark all as read
DELETE	/api/notifications/:id	Delete notification

Method	Endpoint	Description
GET	/api/tags	Get all tags
POST	/api/tags	Create tag (admin)
PUT	/api/tags/:id	Update tag (admin)
DELETE	/api/tags/:id	Delete tag (admin)
GET	/api/tags/:id/events	Get events by tag

5.3 Technology Stack Overview

Table 5.2: Technology Stack Overview

Layer	Technology	Version
Frontend	React.js	18.3.1
Frontend Build	Vite	5.3.4
Backend	Node.js + Express.js	4.19.2
Real-time	Socket.io	latest
Authentication	JWT	—
Database	MySQL	8.0
Styling	Bootstrap	5.3.3
HTTP Client	Axios	1.7.2

5.4 Performance Metrics

- **Notification Latency:** Real-time notifications delivered in $<$100 ms via WebSocket
- **API Response Time:** Average API response time $<$300 ms
- **Page Load Time:** Initial page load $<$2 seconds
- **Concurrent Users:** Supports 50+ concurrent WebSocket connections
- **Database Query Time:** Average query execution $<$100 ms with proper indexing

Figure 5.1: Input of the Eventbooking Dashboard

INPUT:

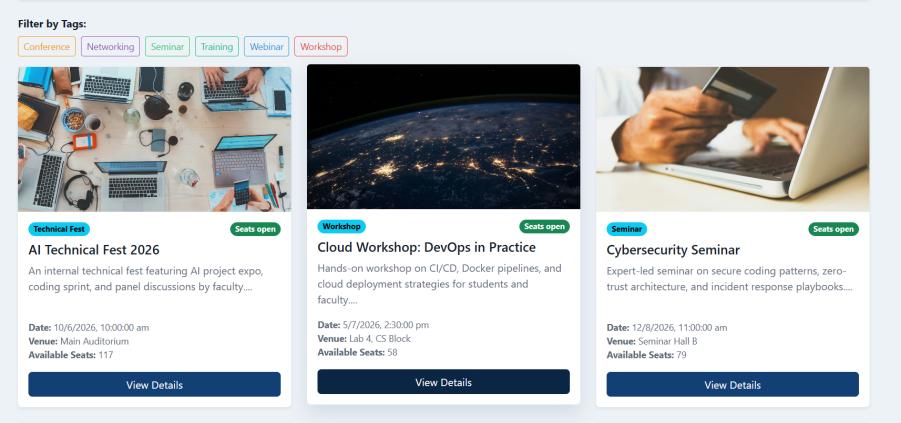

Figure 5.1 represents the primary input interface of the Tech Event Ticket Booking system, where the user-driven data entry process begins. The input layer is designed to capture a variety of parameters, starting with user authentication credentials and profile details. Once logged in, the system accepts inputs in the form of search queries and tag-based filters, allowing users to narrow down events by category, such as ”Workshop” or ”Seminar.” As shown in the diagram, the input phase is critical for data integrity, as it handles the selection of specific events, the number of tickets required, and the submission of payment verification details, all of which are processed to trigger the subsequent booking logic.

OUTPUT:

Figure 5.2: Output of booking Transaction details

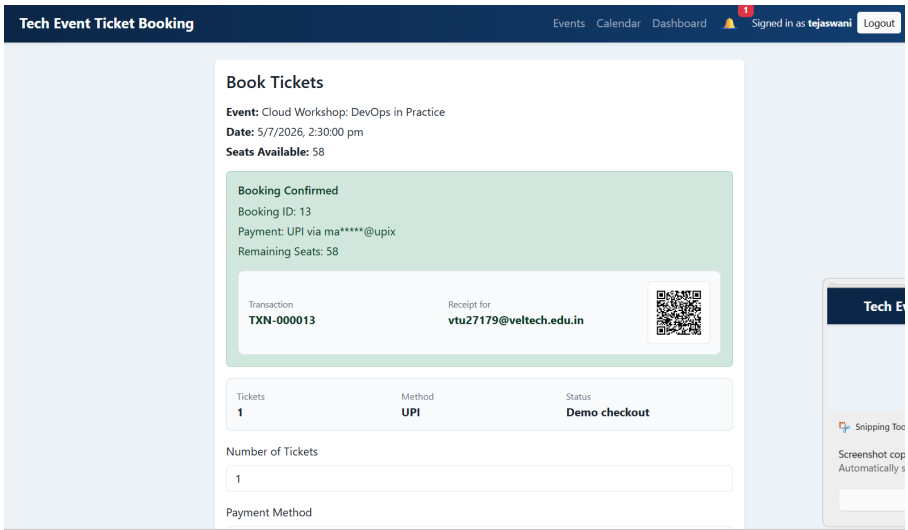

Figure 5.2 displays the output results generated by the system after successful data processing. The output layer is responsible for translating backend logic into human-readable information, primarily through the generation of dynamic booking confirmations and digital tickets. As seen in the figure, the primary output includes a detailed event summary, a unique transaction ID, and a QR code for check-in purposes. This stage ensures that the user receives immediate visual feedback, confirming that their seat has been reserved and their data has been successfully committed to the MySQL database.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Tech Event Ticket Booking system has been successfully developed as a comprehensive full-stack web application providing an effective solution for managing technical events with a user-friendly interface and powerful administrative controls.

Key achievements include:

1. **Complete Event Management System:** Users can browse, filter, and book event tickets seamlessly.
2. **Real-time Notification System:** WebSocket integration enables instant updates on bookings and new events.
3. **Flexible Tag System:** Event categorisation with tags allows better event discovery.
4. **Security Implementation:** JWT-based authentication with role-based access control.
5. **Responsive Design:** Application works seamlessly across desktop and mobile devices.
6. **Scalable Architecture:** Modular design allows future feature additions and scaling.

6.2 Future Enhancements

1. **Payment Gateway Integration:** Integrate with Stripe or PayPal for secure online payments.
2. **Advanced Analytics Dashboard:** Provide admins with insights on event popularity, attendance, and revenue.
3. **Calendar Synchronisation:** Allow users to sync bookings with Google Calendar or Outlook.
4. **Email Reminders:** Automated email reminders before events.
5. **Mobile App:** Native mobile application using React Native.
6. **QR Code Generation:** Generate QR codes for ticket check-in.
7. **User Reviews and Ratings:** Allow users to review and rate events.
8. **Event Recommendations:** AI-based recommendation system based on user history.
9. **Multi-language Support:** Support for multiple languages.
10. **Dark Mode:** Theme support for better user experience.
11. **Accessibility**

: WCAG compliance for users with disabilities.

Database Optimisation: Implement caching with Redis for improved performance.

Load Balancing: Setup for handling increased traffic.

Analytics Integration: Track user behaviour with Google Analytics.

Community Features: Discussion forums and event forums.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	In-depth engineering knowledge	WK4, WK5	Knowledge of React.js, Express.js, MySQL, authentication, payment validation, OTP verification, and UI design.
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability, security, booking limits, seat availability, and ticket validation requirements.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, API structure, state handling, and database design are analysed and implemented.
WP4	Familiarity	Partially new problems	WK4	Real-time updates, notification handling, receipt generation, and OTP verification add engineering complexity.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking validation, RBAC, receipt generation, and QR-code ticket logic are designed from scratch.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Supports students, faculty, organizers, and admins through different dashboards and access levels.
WP7	Interdependence	System-level integration	WK3, WK5	Frontend, backend API, database, email, WebSocket, and payment verification integrated as one system.

Table 7.1: Mapping of the Project to Complex Engineering Problem

The table 1.1 Tech Event Ticket Booking System is categorized as a Complex Engineering Problem (CEP) because it requires the integration of diverse technical disciplines and the resolution of conflicting requirements. Addressing attributes from WP1 to WP7, the project demands in-depth knowledge of full-stack development, including React.js for dynamic interfaces and Node.js with MySQL for robust backend data management.

7.2 Mapping to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Multiple technologies	React.js, HTML, CSS, Bootstrap, Node.js, Express.js, MySQL, JWT, Socket.io, Nodemailer, QR code, and PDF generation are used.
EA2	Level of Interactions	Complex module interaction	Booking, payment, OTP, receipt, notifications, and database changes interact closely with each other.
EA3	Innovation	Modern technologies	Uses hooks, protected routing, real-time notifications, email reminders, and QR-based ticket receipts.
EA4	Consequences	Impact on users	Simplifies ticket booking, reduces manual work, improves event communication, and organises booking records.
EA5	Familiarity	Beyond standard CRUD	Includes real-time features, OTP verification, automated reminders, PDF receipts, QR codes, and admin exports.

Table 7.2: Mapping of the Project to Complex Engineering Activities

The table 7.2 development of the Tech Event Ticket Booking System is characterized as a complex engineering activity due to the diverse range of resources and technologies required to bridge the gap between frontend user experience and backend data integrity. The project involves high-level interaction between independent modules—such as real-time WebSocket notifications, JWT security layers, and automated PDF generation—requiring a cohesive architectural design.

7.3 SDG Mapping

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Improves access to technical learning by allowing students and faculty to discover, register for, and receive updates about workshops, seminars, webinars, and conferences.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates digital innovation using full-stack architecture, MySQL, JWT, Socket.io real-time notifications, and tag-based event categorisation.
SDG 10	Reduced Inequalities	Online booking reduces dependence on manual registration, giving all authenticated users equal access to event details and ticket availability.
SDG 11	Sustainable Cities and Communities	Supports campus community participation by reducing paper-based ticketing, improving venue planning, and enabling timely communication.

Table 7.3: Mapping of the Project to Sustainable Development Goals

The table 7.3 Tech Event Ticket Booking System is designed to align with several United Nations Sustainable Development Goals (SDGs) by leveraging technology to improve academic engagement and infrastructure efficiency. By providing a centralized platform for workshops and seminars, the project directly supports SDG 4 (Quality Education), ensuring students have seamless access to supplemental learning opportunities.

REFERENCES

[1] Express.js Official Documentation. Available: https://expressjs.com/

[2] React.js Documentation. Available: https://react.dev/

[3] Socket.io Documentation. Available: https://socket.io/docs/v4/

[4] MySQL Documentation. Available: https://dev.mysql.com/doc/

[5] JWT Introduction. Available: https://jwt.io/introduction

[6] Node.js Best Practices. Available: https://github.com/goldbergyoni/nodebestpractices

[7] RESTful API Design Guidelines. Available: https://restfulapi.net/

[8] Bootstrap Documentation. Available: https://getbootstrap.com/docs/

[9] Eventbrite. Available: https://www.eventbrite.com/

[10] Meetup. Available: https://www.meetup.com/

[11] Ticketmaster. Available: https://www.ticketmaster.com/

[12] Axios Documentation. Available: https://axios-http.com/docs/intro

[13] React Router Documentation. Available: https://reactrouter.com/

[14] Bcryptjs Documentation. Available: https://github.com/dcodeIO/bcrypt.js

[15] Nodemailer Guide. Available: https://nodemailer.com/about/